

# AI Incident Learning and Prevention Agent

Institutional memory for every Live Network Intervention (LNI)

AGENTIC AI

RAG

API

Level: Medium

Squad of 3 to 5

## The situation

Every Live Network Intervention (LNI) leaves a trail: the change ticket, the MOP that was followed, execution notes, problems faced, the RCA and the fix. Months later, another engineer plans a similar change and hits the same problem. The answer already exists, but it is buried in free-text tickets, written in different words by different people.

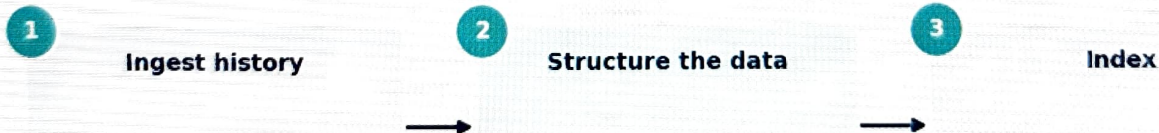
- Past tickets are searched by title or keyword only, so reworded incidents are never found.
- The same failure is described in inconsistent language, with no shared vocabulary.
- Learning stays locked in one ticket and never becomes organisational memory.
- Result: repeat outages and rollbacks caused by problems already solved.

## Your mission

Build an agent that ingests the LNI history of the last few months and keeps it as searchable institutional memory. It must (a) warn engineers of known pitfalls and recommend validation steps BEFORE a planned LNI, (b) find technically similar past cases DURING an incident, and (c) write a verified learning back AFTER resolution.

**Who will use it:** Change engineers, NOC and SNOC teams, delivery leads.

## How the solution should work



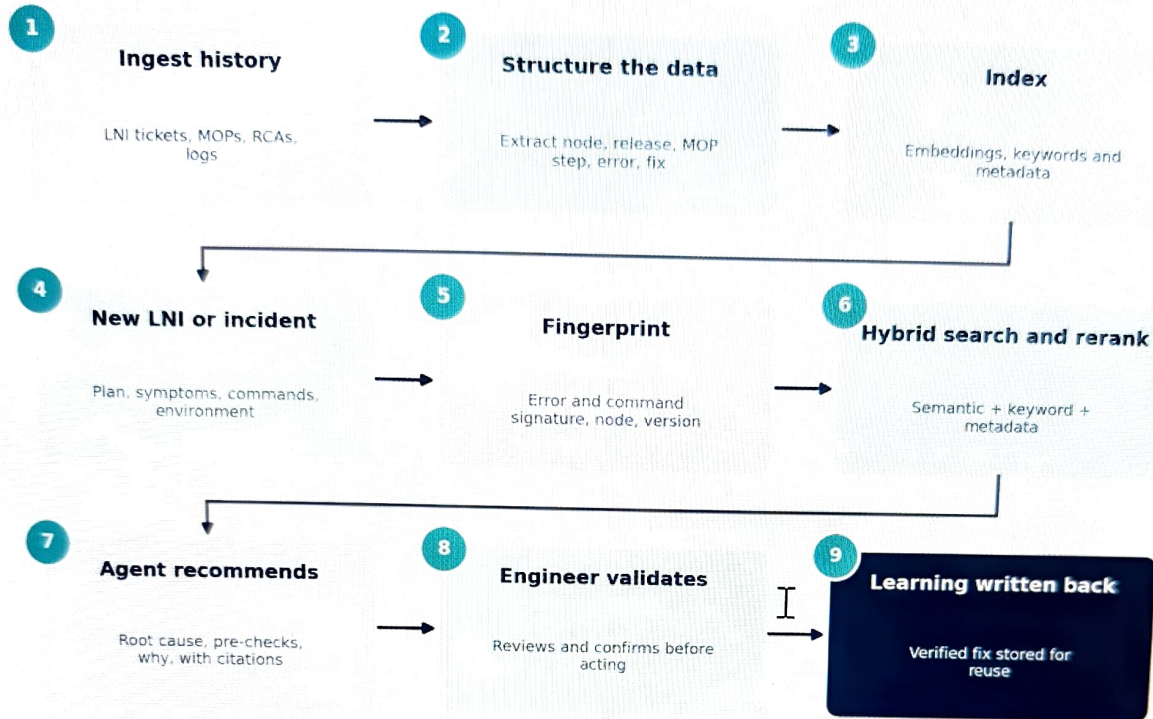
Search



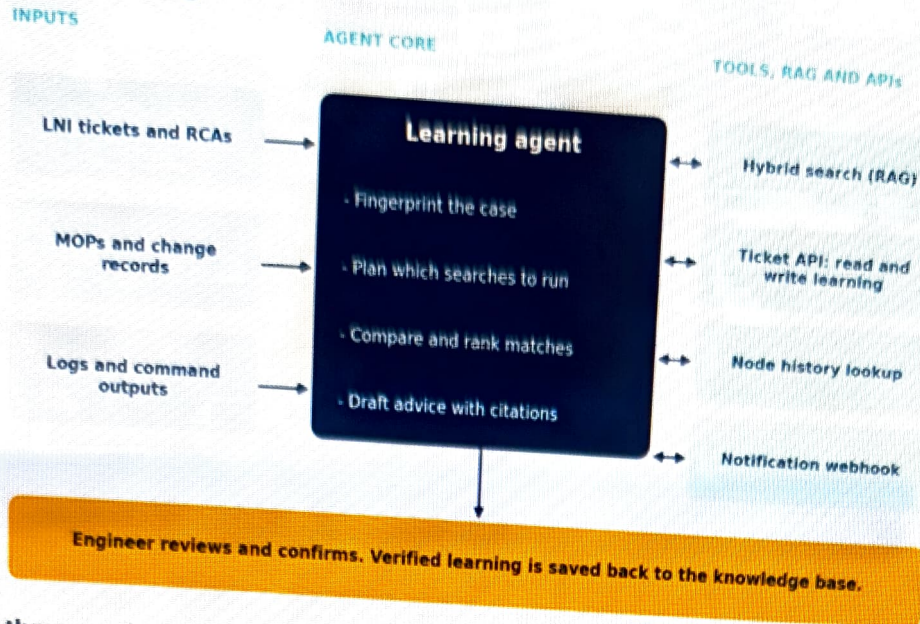
ess, place your favorites here on the favorites bar. Looking for your favorites? [Check your profiles](#)

must (a) warn engineers of known pitfalls and recommend validation steps BEFORE a planned LNI, (b) find technically similar past cases DURING an incident, and (c) write a verified learning back AFTER resolution.  
**Who will use it:** Change engineers, NOC and SNOC teams, delivery leads.

**How the solution should work**



## Architecture at a glance



## The three angles your solution must cover

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Agentic AI</b> | <ul style="list-style-type: none"> <li>The agent plans its own searches: by symptom, by node history, by MOP step. It re-queries when confidence is low.</li> <li>It asks the engineer a clarifying question when node, release or MOP step is missing.</li> <li>It calls tools (search, node history, ticket write-back) and checks that every claim has a citation before answering.</li> <li>It never executes changes. It surfaces history and the engineer decides.</li> </ul> |
| <b>RAG</b>        | <ul style="list-style-type: none"> <li>Hybrid retrieval: semantic search for meaning, keyword search for exact error codes and commands, metadata filters for vendor, node type and release, then a reranker.</li> <li>Chunk by field (root cause, resolution, MOP step) so answers cite the right part.</li> <li>Show a confidence score and the source records for every match.</li> <li>Say 'no verified match found' instead of inventing a fix.</li> </ul>                     |
| <b>API</b>        | <ul style="list-style-type: none"> <li>REST endpoints: POST /ingest, POST /match, POST /recommend, POST /feedback (write back verified learning).</li> <li>Mock ticketing API to read incidents and write learnings.</li> <li>Webhook to Teams or email for high-risk matches before a planned LNI.</li> <li>Auto-generated OpenAPI docs so other tools can call the agent.</li> </ul>                                                                                              |

## Data you will work with

- About 100 synthetic LNI records (JSON or CSV) including reworded duplicates of the same underlying problem.
- 5 to 10 sample MOP documents and a few log snippets.
- A test set of 20 new incidents with the known correct past match.

### Sample scenario

Planned LNI: 'Software upgrade on packet core gateway CMG-12 using MOP-UPG-04'. The agent should surface two past LNIs where interface MTU and BFD timers reverted to defaults after an upgrade, recommend 'verify MTU and BFD timers before re-enabling sessions', and cite both records with confidence scores.

# What to deliver

## Must have

- Ingest and index the dataset
- Hybrid search with reranking
- Recommendation with citations and confidence
- REST API and a simple UI
- Write-back of a verified learning

## Stretch goals

- Repeat-failure trend dashboard
- Weekly digest of top recurring issues
- Pre-change risk score
- LLM-based field extraction from raw tickets

Also hand in: a short architecture diagram, a README, and a 5-slide pitch deck for management (problem, solution, demo, results, next steps).

# Suggested tech stack

| Layer                       | Suggested options                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------------------|
| Language and API            | Python 3.11, FastAPI, Pydantic                                                                          |
| LLM                         | Organiser-approved endpoint, or a local open model through Ollama (Llama 3, Mistral) for sensitive data |
| Agent orchestration         | LangGraph or LlamaIndex agents, or a plain tool-calling loop                                            |
| Embeddings and vector store | sentence-transformers (bge-small or MiniLM); ChromaDB, FAISS or Qdrant                                  |
| Keyword search and rerank   | rank_bm25 or OpenSearch; cross-encoder reranker (bge-reranker)                                          |
| Data handling               | pandas, python-docx, pypdf                                                                              |
| UI                          | Streamlit or Gradio                                                                                     |
| Quality and delivery        | pytest, Git, Docker                                                                                     |

These are suggestions. Choose what your squad can build fastest, and explain your choice in the pitch.

access, place your favorites here on the favorites bar. Looking for your favorites? [Check your profiles](#)



Ask Copilot ▾



3

of 3



## Quality and delivery

pytest, Git, Docker

These are suggestions. Choose what your squad can build fastest, and explain your choice in the pitch.

## Ground rules

- Use only synthetic or sanitised data. No real customer data.
- A human validates before any action. The agent is read-only unless this statement says otherwise.
- Every claim must trace back to a source. Say 'not found' instead of guessing.
- Use only AI tools and endpoints approved by the organisers.

## Suggested build plan

### Stage

### Focus

**Sprint 1 (Day 2 afternoon)**

Load and understand the data, working retrieval or parsing, skeleton API

**Sprint 2 (Day 3 morning)**

Agent loop with tools and guardrails, UI, integration

**Final build (Day 3 early afternoon)**

Test on the scenarios, measure accuracy, prepare the demo and pitch deck

## How it will be judged

| Criterion                                     | Weight |
|-----------------------------------------------|--------|
| Working end-to-end demo                       | 30%    |
| Agentic design: tools, planning, guardrails   | 20%    |
| RAG quality: retrieval accuracy and citations | 20%    |
| API and integration                           | 10%    |
| Reliability and safety                        | 10%    |
| Pitch and clarity                             | 10%    |

Evaluators will also observe your approach, your pace and how well you understand the concepts while you build.



Search



ENG  
IN